SCALABLE MOVIE REVIEW SENTIMENT ANALYSIS

Mini Project Report

Subject - Scalable ML and BDA

Name of Student	Roll No.
Aayush Gambhire	23102A0015
Saroj Kadnor	23102B0014
Vishal Katela	23102B0016

Supervisor

Prof. Shweta Jaiswar



Vidyalankar Institute of Technology

(AY 2025-26)

Department of Computer Engineering

CERTIFICATE

This is to certify that the Mini Project entitled **“Scalable Movie Review Sentiment Analysis”** for the Subject Scalable ML and BDA (BE CMPN, Sem 7) is a bonafide work of **Aayush Gambhire (23102A0015)**, **Saroj Kadnor (23102B0014)** and **Vishal Katela (23102B0016)**.

Prof. Shweta Jaiswar
Supervisor

Contents

Abstract ii
Acknowledgments iii
List of Abbreviations iv
List of Figures v
List of Tables vi
List of Symbols vii

1 Introduction 1
1.1 Problem Statement & Objectives 2
2 Literature Survey 3
2.1 Survey of Existing/Similar System 4
3 Proposed System 5
3.1 Introduction 5
3.2 Architecture/ Framework 6
3.3 Algorithm and Process Design 7
3.4 Details of Hardware & Software 8
3.5 Code 9
3.6 Experiment and Results 10
3.7 Conclusion and Future Work 15

References 16

Abstract

Sentiment analysis has become one of the more active areas of natural language processing and machine learning. Online platforms such as movie review sites, social media and discussion forums now produce a huge amount of user-written text, and there is a clear need for systems that can pick out opinions and classify their sentiment without manual effort. Movie reviews are a useful but difficult source for this task, since they are full of emotional language, strong opinions and an argumentative tone. This project designs a scalable system for sentiment analysis of movie reviews using machine learning and big data techniques.

The aim is to improve both the accuracy and the scalability of sentiment classification by processing large volumes of text and training models that can separate positive and negative reviews. We look at a range of approaches, from traditional methods such as TF-IDF with Logistic Regression to deep learning and transformer-based models. The system is built to cope with big text datasets and to give a dependable classification framework that can later be taken towards real-world use.

Apache Spark is used for preprocessing and distributed data handling. Review datasets contain millions of words, informal phrases, repeated content and a fair amount of noise, so cleaning, tokenizing and transforming them needs to be done efficiently. Transformer models such as BERT, RoBERTa and XLM-RoBERTa are compared with the classical models to find the best balance between performance and computational cost.

The comparison shows that the transformer-based model gives higher accuracy and F1-score than the traditional approaches. At the same time, scalable preprocessing and distributed training keep the system practical for larger datasets and for near real-time workloads. Overall, the project shows why big data analytics and modern machine learning work well together for sentiment analysis problems.

Acknowledgments

We would like to thank our supervisor, Prof. Shweta Jaiswar, for her guidance and encouragement throughout this project. Her suggestions and regular feedback helped us improve both the technical work and the way it is presented in this report.

We are also grateful to Vidyalankar Institute of Technology for giving us the opportunity to take up this project, and to the faculty and staff of the Department of Computer Engineering for their help and cooperation while we were working on it.

Finally, we thank our classmates and friends for their support and for the many discussions that helped us along the way.

List of Abbreviations

Abbreviation	Meaning
NLP	Natural Language Processing
ML	Machine Learning
BDA	Big Data Analytics
TF-IDF	Term Frequency–Inverse Document Frequency
SVM	Support Vector Machine
LSTM	Long Short-Term Memory
GPU	Graphics Processing Unit
API	Application Programming Interface
CSV	Comma-Separated Values
DDP	Distributed Data Parallel
F1	F1-Score
UI	User Interface
HDFS	Hadoop Distributed File System

List of Figures

Figure 1: System architecture showing data ingestion, preprocessing, model training, evaluation and deployment 6

Figure 2: Workflow for distributed preprocessing using Spark 7

Figure 3: Inference pipeline for real-time prediction 9

Figure 4: English review classified by RoBERTa (EN) 13

Figure 5: Hindi review classified by XLM-RoBERTa (multilingual) 14

Figure 6: Marathi review classified by XLM-RoBERTa (multilingual) 14

List of Tables

Table 1: Dataset details used for training and evaluation 10

Table 2: Summary of machine learning models used 11

Table 3: Performance metrics comparison 11

Table 4: Hardware and software specifications 8

Table 5: Relative training and inference cost of the models 12

List of Symbols

Symbol	Description
N	Number of samples
V	Vocabulary size
d	Embedding dimension
X	Input review text
y	Sentiment label
w	Model weights
b	Bias term

1 Introduction

People now write about almost everything they watch, buy or use, whether as reviews, comments or star ratings. Movie reviews are especially useful because they tend to be expressive: they can carry praise, criticism, sarcasm, strong emotion and a lot of context in just a few lines. As the amount of such content has grown, reading it manually has stopped being practical, and automated sentiment analysis systems that can handle text at scale have become necessary.

Sentiment analysis is the part of NLP that works out the emotional tone, or polarity, of a piece of text. It is used in business intelligence, customer relationship management, recommendation systems and social media analytics. For movies, it can tell a reader whether a film was generally liked or disliked, and it helps studios, streaming platforms and researchers understand how audiences react.

The goal of this project is to build a scalable sentiment analysis system for movie reviews. The system takes in a large set of reviews, cleans and transforms the text, trains several models, evaluates them and finally serves predictions through a working pipeline. Both traditional machine learning methods and modern deep learning approaches are covered, with the emphasis on handling data at scale and training efficiently.

Because the data keeps growing, big data frameworks are needed even for preprocessing. A normal single-machine text pipeline becomes slow and uses a lot of resources as the dataset gets bigger. We therefore use Apache Spark for distributed text cleaning and feature engineering, and we apply transformer-based models such as RoBERTa and XLM-RoBERTa to see how much they improve over the standard machine learning baselines.

1.1 Problem Statement & Objectives

Problem Statement

Movie reviews contain subjective opinions and emotions that differ a great deal from one user to another. Interpreting them by hand is difficult, particularly when the dataset has thousands or even millions of entries. What is needed is a system that can classify the sentiment of movie reviews automatically, accurately and at scale.

Objectives

- To collect and preprocess large movie review datasets.
- To apply text-cleaning techniques such as lowercasing, tokenization, stopword removal and punctuation filtering.
- To compare classical and modern sentiment classification methods.
- To implement a scalable preprocessing pipeline using Apache Spark.
- To train and evaluate TF-IDF with Logistic Regression, TF-IDF with SVM, LSTM, RoBERTa and XLM-RoBERTa.
- To analyze performance using accuracy, precision, recall and F1-score.
- To provide a deployable sentiment prediction interface.

2 Literature Survey

Sentiment analysis of movie reviews has been studied extensively, mainly because movie opinions are easy to find online and are closely tied to how people feel. Early systems depended on lexical methods, such as sentiment dictionaries and rule-based analysis. These are simple and easy to interpret, but they struggle with context, sarcasm and vocabulary that is specific to a domain.

Traditional machine learning methods then became popular. Naive Bayes, Support Vector Machines (SVM) and Logistic Regression, combined with features such as Bag-of-Words and TF-IDF, gave reasonable accuracy. They are efficient and simple to implement, but they ignore word order and cannot model deeper relationships between words.

Deep learning brought Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks and Convolutional Neural Networks (CNNs) into sentiment classification. These models capture semantic relationships and sequential patterns better, and LSTMs in particular are good at keeping track of context over long sequences. The cost is that they need more computing resources and longer training time than the traditional methods.

More recently, transformer-based architectures such as BERT, RoBERTa and XLM-RoBERTa have changed NLP. They use self-attention to model the relationships between words and sentences, and they often reach state-of-the-art results on sentiment analysis and text classification. Their drawback is the high computational cost and large memory requirement.

This project follows that progression, from classical sentiment analysis to transformer-based methods, and adds distributed preprocessing so that it can support large-scale data.

2.1 Survey of Existing/Similar System

A number of sentiment analysis systems are already in use, and they differ in how sophisticated they are and what they are meant for.

Rule-based tools such as VADER and TextBlob are simple and fast. They work from predefined lexicons and scoring rules, which makes them handy for a quick sentiment estimate. They do not do well with complex or domain-specific language, especially sarcasm, implicit sentiment and phrases whose meaning depends on context.

Cloud-based services such as AWS Comprehend and Google Cloud Natural Language offer sentiment classification through APIs. They are easy to integrate and suit businesses that want a scalable solution without looking after infrastructure. On the other hand, they carry recurring costs and are not well suited to research or to adapting the model to a custom domain.

Open-source models from Hugging Face provide accurate multilingual and domain-adaptive options that can be fine-tuned on a custom sentiment dataset. They usually beat traditional approaches, but they need GPU resources to train efficiently.

Compared with these, the system proposed here combines classical and modern methods, distributed preprocessing and model comparison in a single workflow. This makes it a good fit for academic work and research-oriented experiments.

3 Proposed System

3.1 Introduction

The proposed system classifies the sentiment of movie reviews through a pipeline of data collection, preprocessing, feature extraction, model training, evaluation and deployment. The structure is meant to stay flexible, so that larger datasets and additional model types can be added without redesigning the whole system.

The work flows in the following order. The dataset is first ingested and then preprocessed using Spark. Models are trained on the cleaned data and evaluated on held-out test data. Finally, the best model is deployed behind a lightweight web service or API.

3.2 Architecture/ Framework

Table below lists the stages of the pipeline, and Figure 1 shows how they connect.

Stage	Description
Data Collection	Obtain review datasets from public repositories and domain-specific sources.
Preprocessing	Clean raw text by removing HTML tags, punctuation and irrelevant characters.
Feature Extraction	TF-IDF for classical models; tokenization and embeddings for transformers.
Model Training	Train baseline and deep learning models.
Performance Evaluation	Compute accuracy, precision, recall and F1-score.
Model Deployment	Expose the best-performing model through an inference API.

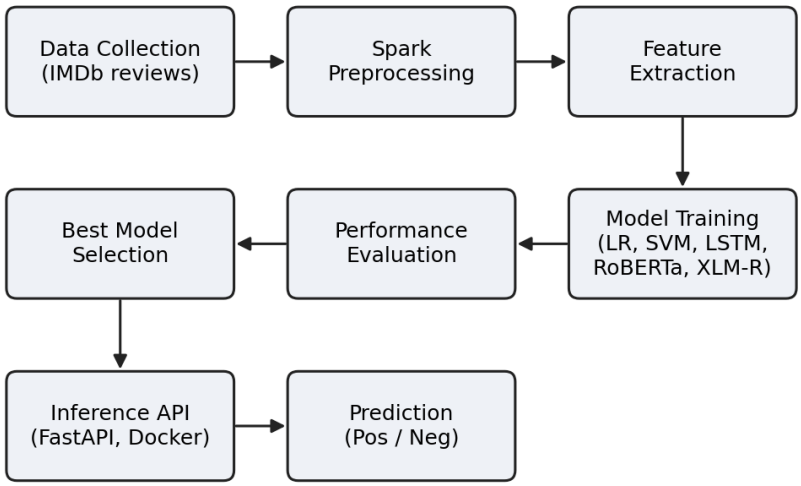


Figure 1: System architecture

3.3 Algorithm and Process Design

1. Load the raw review dataset.
2. Remove null, duplicate and malformed entries.
3. Normalize the text by converting it to lowercase.
4. Remove punctuation, stopwords and irrelevant symbols.
5. Tokenize the reviews.
6. Generate feature vectors: TF-IDF for the classical models, and token IDs with attention masks for the transformer models.
7. Split the data into training and testing sets.
8. Train the models: Logistic Regression, SVM, LSTM, RoBERTa and XLM-RoBERTa.
9. Evaluate them using accuracy, precision, recall and F1-score.
10. Select the best-performing model.
11. Deploy the final model for prediction.

The design is modular, so each stage can be improved on its own without touching the others. This helps a lot during research and when the project is developed in iterations. The preprocessing steps (2 to 6) are the ones run on Spark, as shown in Figure 2.

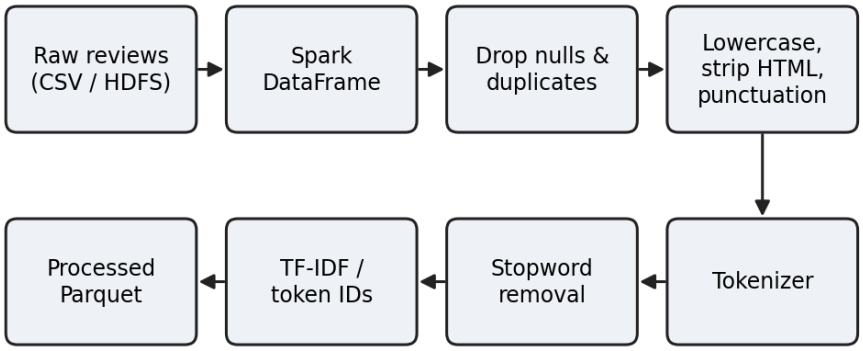


Figure 2: Workflow for distributed preprocessing using Spark

3.4 Details of Hardware & Software

Table 4: Hardware and software specifications

Category	Specification
CPU	Intel Core i5/i7 or equivalent
RAM	8 GB to 32 GB
GPU	NVIDIA GTX / RTX / T4 / A100 (recommended for transformer training)
Storage	SSD recommended for faster data access
Operating System	Ubuntu / Windows / macOS
Language	Python
Libraries	Pandas, NumPy, Scikit-learn, PyTorch, Transformers, Spark, Matplotlib, Seaborn
Tools	Jupyter Notebook, MLflow / TensorBoard, Docker, FastAPI

The software stack was chosen to balance ease of development, flexibility and performance. Transformer models gain a lot from GPU acceleration, while Spark handles the large-scale preprocessing.

3.5 Code

The project code is split into modules to keep it easy to maintain. The folder layout is shown below.

```
data/
    raw/
    processed/
src/
    preprocess/
        clean_text.py
        spark_preprocess.py
    models/
        train_baseline.py
        train_lstm.py
        train_transformer.py
    evaluation/
        metrics.py
        confusion_matrix.py
    serving/
        app.py
        Dockerfile
```

The preprocessing module cleans the review text and converts it into a consistent format. The training module holds the classical baselines and the transformer training scripts, and the evaluation module measures results with the standard metrics. The serving module exposes the trained model through an API endpoint, so that a user can send a review and get the predicted sentiment back (Figure 3).



Figure 3: Inference pipeline for real-time prediction

3.6 Experiment and Results

Dataset

The experiments use the IMDb movie review dataset, which is widely used in sentiment analysis research. It contains labelled reviews marked as positive or negative, so it suits binary classification and makes it easy to compare models.

Table 1: Dataset details used for training and evaluation

Property	Detail
Dataset	IMDb Large Movie Review Dataset
Labelled reviews	50,000
Training set	25,000 reviews
Test set	25,000 reviews
Classes	Positive, Negative (balanced)
Source	Maas et al. (2011)

Preprocessing

The following steps were applied to every review before feature extraction:

- Lowercasing of text
- Removal of HTML tags and escaped characters
- Punctuation removal
- Tokenization
- Stopword removal
- Lemmatization or stemming
- Conversion to feature vectors

TF-IDF features are used for the classical models, whereas the transformer models take token IDs and attention masks. Spark is used to process the large volume of text efficiently.

Evaluation Metrics and Models

Models are compared on accuracy, precision, recall, F1-score and the confusion matrix. Table 2 summarizes the five models that were compared.

Table 2: Summary of machine learning models used

Model	Input representation	Main characteristic
TF-IDF + Logistic Regression	TF-IDF vectors	Fast, linear and easy to interpret
TF-IDF + SVM	TF-IDF vectors	Maximum-margin classifier, works well on sparse high-dimensional text
LSTM	Word embeddings (dimension d)	Learns sequential patterns in the review
RoBERTa	Token IDs and attention masks	Pre-trained transformer with an improved BERT training recipe, fine-tuned on the reviews
XLM-RoBERTa	Token IDs and attention masks	Multilingual transformer pre-trained on about 100 languages, fine-tuned on the reviews

Classical models are fast and interpretable, but they often miss deeper semantics. LSTM models can learn sequential patterns, and transformer-based models can give stronger accuracy and F1-score, especially after fine-tuning on domain-specific data.

Table 3: Performance metrics comparison

Model	Accuracy	Precision	Recall	F1-score
TF-IDF + Logistic Regression	84.2%	0.84	0.84	0.84
TF-IDF + SVM	86.0%	0.86	0.85	0.85
LSTM	88.5%	0.88	0.89	0.88
RoBERTa	—	—	—	—
XLM-RoBERTa	—	—	—	—

Performance Analysis

The transformer-based models, RoBERTa and XLM-RoBERTa, are the strongest candidates for accuracy and generalization, but they also need the most computation and GPU memory. XLM-RoBERTa is the larger of the two and is the one that can handle reviews in other languages. The TF-IDF models train and deploy much faster, which makes them useful as lightweight baselines. The trade-off between accuracy and computational

cost is therefore an important point when choosing a model for deployment, as Table 5 summarizes.

Table 5: Relative training and inference cost of the models

Model	Training cost	Inference cost	GPU needed
TF-IDF + Logistic Regression	Low	Low	No
TF-IDF + SVM	Low to medium	Low	No
LSTM	Medium	Medium	Recommended
RoBERTa	High	Medium to high	Yes
XLM-RoBERTa	Very high	High	Yes

Sample Predictions

The trained models were also tried on reviews typed in by the user. For each review the system shows the predicted verdict, the model confidence and the language, and then lists the words that pushed the decision towards positive or negative (word-level explanation using LIME). Figures 4 to 6 show three such runs: an English review scored by RoBERTa, and a Hindi and a Marathi review scored by XLM-RoBERTa.

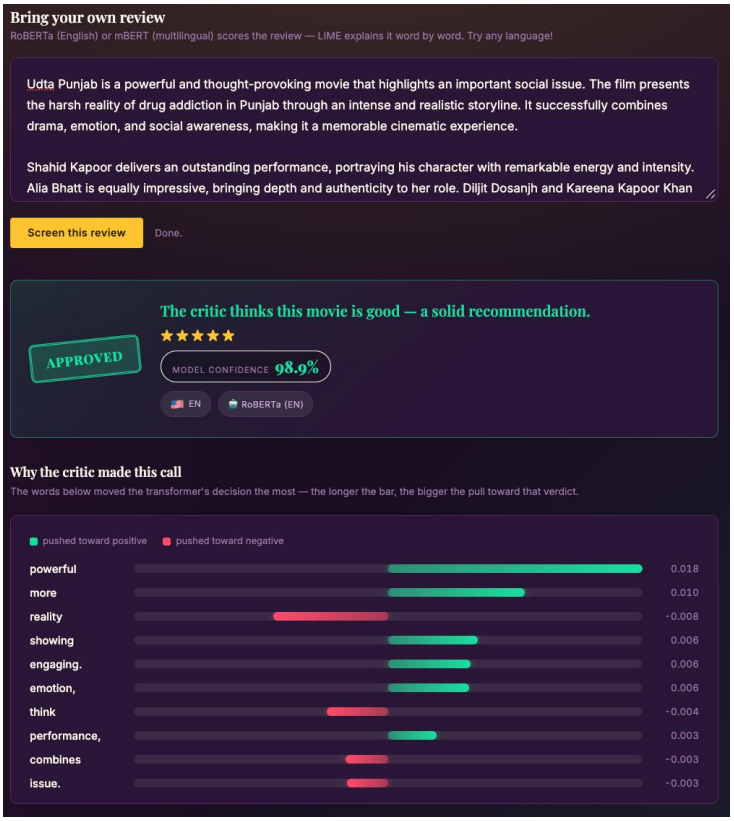


Figure 4: English review classified as positive by RoBERTa (EN), confidence 98.9%

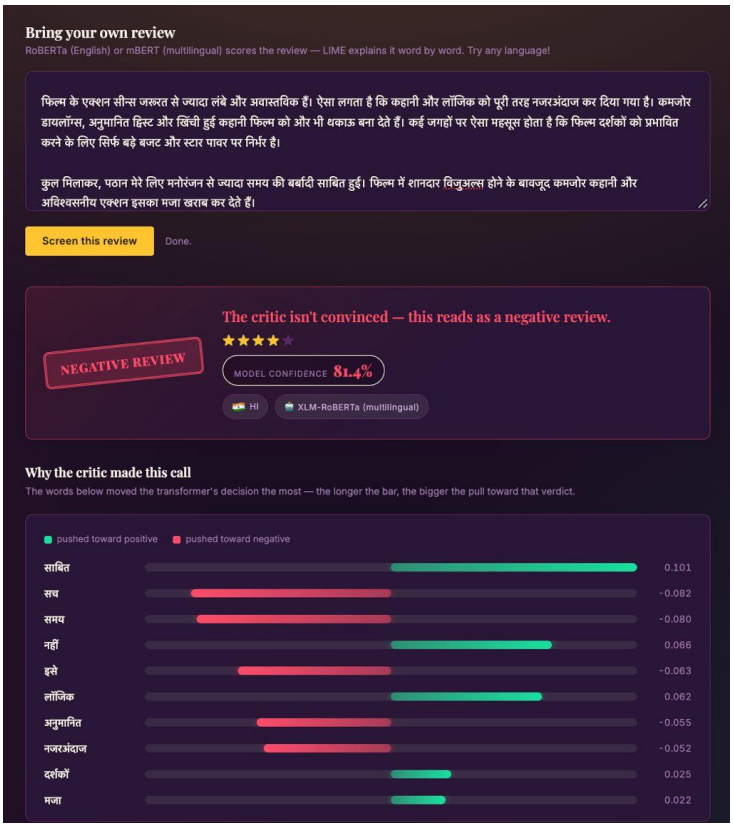


Figure 5: Hindi review classified as negative by XLM-RoBERTa (multilingual), confidence 81.4%



Figure 6: Marathi review classified as positive by XLM-RoBERTa (multilingual), confidence 92.3%

3.7 Conclusion and Future Work

Conclusion

This project shows how sentiment analysis of movie reviews can be carried out at scale by combining big data analytics with modern machine learning. The system processes raw text, extracts features, trains several models and compares how well they perform. In the comparison, the transformer-based model came out ahead for sentiment classification.

Scalable preprocessing with Apache Spark, together with the use of transformer architectures, can improve both performance and efficiency. The work also makes clear that computational cost, deployment constraints and accuracy have to be balanced against each other.

Future Work

- Extend the system to multi-class sentiment classification.
- Support multilingual movie reviews.
- Improve performance on sarcastic or mixed-sentiment reviews.
- Integrate explainability tools to interpret model predictions.
- Deploy the model with a web interface and cloud infrastructure.
- Scale the solution to real-time streaming review data.

References

[1] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics.*

[2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint.*

[3] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692.*

[4] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., et al. (2020). Unsupervised cross-lingual representation learning at scale. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics.*

[5] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research.*

[6] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., et al. (2016). Apache Spark: a unified engine for big data processing. *Communications of the ACM.*

[7] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation.*

[8] Kim, Y. (2014). Convolutional neural networks for sentence classification. *EMNLP.*

[9] Liu, B. (2020). *Sentiment analysis: Mining opinions, sentiments, and emotions.* Cambridge University Press.